

Gardener F5 Application Load Balancer Extension

Part 5 - Deployment Managers, Typed CMP Clients, Ownership, and Validation

Implementation-grade design for CMP reconciliation, asynchronous operations, retries, ownership, adoption, deletion safety, typed API models, validation, and error classification.

Architecture and Low-Level Design

Document role: This part defines the provider-facing execution boundary shared by the Service and Ingress designs in Parts 3 and 4. It does not redefine Kubernetes semantics; it specifies how an already validated desired-state graph is safely converged through CMP.

Table of Contents

5.1 Scope and Design Objectives
5.2 Layered Architecture and Responsibilities
5.3 Stack Deployer Contract
5.4 Resource Deployment Managers
5.5 Dependency Planning and Action Graph
5.6 Typed CMP Client Architecture
5.7 Typed API Models and Mapping
5.8 Asynchronous CMP Operations
5.9 Idempotency, Retries, and Ambiguous Outcomes
5.10 Discovery, Diffing, and Normalization
5.11 Ownership Identity and Metadata
5.12 Adoption and Conflict Handling
5.13 Validation Before Mutation
5.14 Deletion Safety and Finalization
5.15 Error Classification
5.16 Status, Events, Metrics, and Auditability
5.17 Concurrency, Rate Limiting, and Caching
5.18 Security and Credential Handling
5.19 Testing Strategy
5.20 Package and Interface Design
5.21 CMP Endpoint Mapping
5.22 Sequence Diagrams
5.23 Design Decisions
5.24 Implementation Checklist

5.1 Scope and Design Objectives

Parts 3 and 4 produce a complete desired-state stack. Part 5 defines the execution machinery that turns that stack into an observed CMP/F5 resource graph. The central rule is that provider mutation is never performed directly by a Kubernetes controller or a model builder.

5.1.1 Objectives

- Provide one reusable deployment layer for Service and Ingress stacks.
- Make every reconcile idempotent and restart-safe.
- Represent CMP requests and responses with typed Go models rather than unstructured maps.
- Support asynchronous CMP operations without blocking controller workers indefinitely.
- Prevent accidental adoption, update, or deletion of resources owned by another object or cluster.
- Classify failures into retryable, conflict, permanent, authentication, throttling, and unknown-outcome categories.
- Preserve parent resources while reconciling frequently changing child resources such as pool members.
- Produce auditable decisions, stable identifiers, status conditions, Kubernetes Events, and metrics.

5.1.2 Non-goals

- Embedding Kubernetes API reads inside CMP clients.
- Using CMP response objects as the internal desired-state model.
- Blindly retrying every non-2xx response.
- Inferring ownership from display names alone when stronger metadata is available.
- Deleting resources solely because they are absent from one partial list response.

5.2 Layered Architecture and Responsibilities

Deployment and CMP Integration Layers

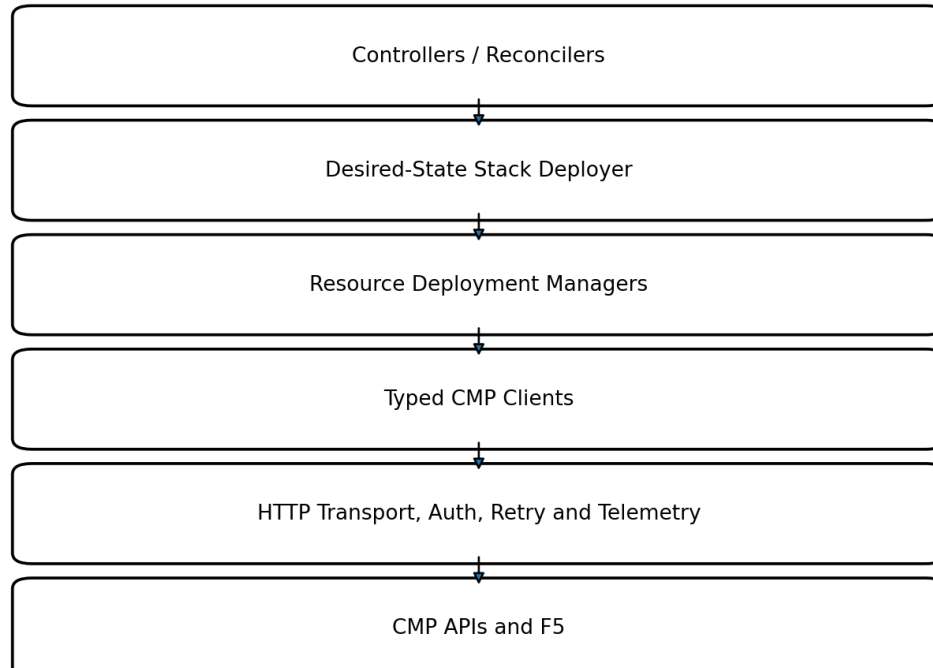


Figure 5-1. Provider-facing layers and dependency direction.

Layer	Owns	Must not own
Controller	Reconcile trigger, input snapshot, final result handling.	CMP endpoint paths, polling loops, resource-specific diffing.
Stack Deployer	Whole-graph plan, dependency order, aggregate result.	Raw HTTP or Kubernetes object parsing.
Resource Manager	Discover, compare, create/update/delete one resource kind.	Authentication and transport implementation.
Typed CMP Client	Typed endpoint operations and response decoding.	Kubernetes semantics or ownership policy decisions.
Transport	HTTP, Ce-Auth/HMAC, deadlines, safe low-level retries, telemetry.	Business-level retries and adoption.
CMP	Authoritative provider state and asynchronous execution.	Kubernetes desired-state intent.

Dependency rule: Dependencies point downward only. Typed clients may be mocked by managers; managers may be mocked by the stack deployer. No lower layer imports controller or Kubernetes reconciliation packages.

5.3 Stack Deployer Contract

Stack Deployment Reconciliation



Figure 5-2. Stack-level deployment reconciliation.

The stack deployer coordinates a complete LoadBalancerStack or IngressStack. It receives an immutable desired graph, creates a plan, executes the plan, and returns a provider result that contains stable CMP IDs, the VIP, readiness, warnings, and classified errors.

```

type StackDeployer interface {
    Deploy(ctx context.Context, desired *model.Stack) (*DeploymentResult, error)
    Delete(ctx context.Context, owner ownership.OwnerRef) (*DeletionResult, error)
}

type DeploymentResult struct {
    ResourceIDs map[model.LogicalID]string
    VIPAddress  string
    Ready       bool
    Warnings    []Warning
    ObservedAt  time.Time
}
  
```

5.3.1 Required Behavior

1. Validate the complete desired stack and reject unsafe plans before mutation.
2. Discover the current owned graph using persisted IDs and ownership lookup keys.
3. Normalize desired and observed values before comparison.
4. Construct a dependency-aware action plan.
5. Execute actions with bounded concurrency and cancellation.
6. Wait only for operations required to establish the next dependency.
7. Verify terminal provider state, not merely successful HTTP submission.
8. Return partial progress safely so the next reconcile can resume.

5.3.2 Atomicity Model

CMP does not expose one transaction covering the full graph. Therefore atomicity is logical, not database-transactional. The deployer uses ordered convergence and compensating cleanup rather than pretending the graph is created atomically. A failure leaves a discoverable, owned partial graph that the next reconcile continues or removes.

5.4 Resource Deployment Managers

Manager	Primary responsibilities	Parent dependencies
LBServiceManager	Ensure top-level service; expose provider ID and lifecycle state.	None.
VIPManager	Ensure VIP/network port; return address and VIP ID.	LB Service.
VirtualServerManager	Ensure protocol/listener, frontend port, default pool binding and certificate association.	LB Service; VIP semantics.
PoolManager	Ensure pool algorithm and parent relation.	Virtual Server.
MemberManager	Bulk or incremental reconciliation of backend members.	Pool; resolved backend_port_id.
MonitorManager	Ensure health monitor configuration and availability policy.	Pool.
RoutingRuleManager	Ensure deterministic host/path rules and pool targets.	Virtual Server and target Pool.
CertificateManager	Upload/deduplicate certificate and bind it to HTTPS Virtual Server.	LB Service and Virtual Server.

5.4.1 Manager Interface Pattern

```
type Manager[D any, O any] interface {
    Discover(ctx context.Context, scope Scope, key ownership.LookupKey) ([]O, error)
    Classify(desired D, observed []O) (MatchResult[O], error)
    Create(ctx context.Context, desired D, meta RequestMeta) (OperationRef, error)
    Update(ctx context.Context, current O, desired D, meta RequestMeta) (OperationRef, error)
    Delete(ctx context.Context, current O, meta RequestMeta) (OperationRef, error)
    Verify(ctx context.Context, ref ResourceRef) (O, error)
}
```

Managers may use specialized methods instead of one generic interface, but all must follow the same lifecycle: discover, ownership-check, compare, mutate, wait, and verify.

5.4.2 Manager Invariants

- A manager never mutates an observed resource until ownership is verified.
- A manager treats HTTP acceptance as submission, not success.
- A manager never creates a duplicate after an unknown outcome without rediscovery.
- A manager reports immutable-field changes to the planner as Replace, not as an unsafe partial patch.
- A manager removes only children whose ownership identity belongs to the desired stack owner.

5.5 Dependency Planning and Action Graph

CMP Resource Dependency Graph and Safe Ordering

Create/update follows arrows; deletion runs in reverse dependency order.

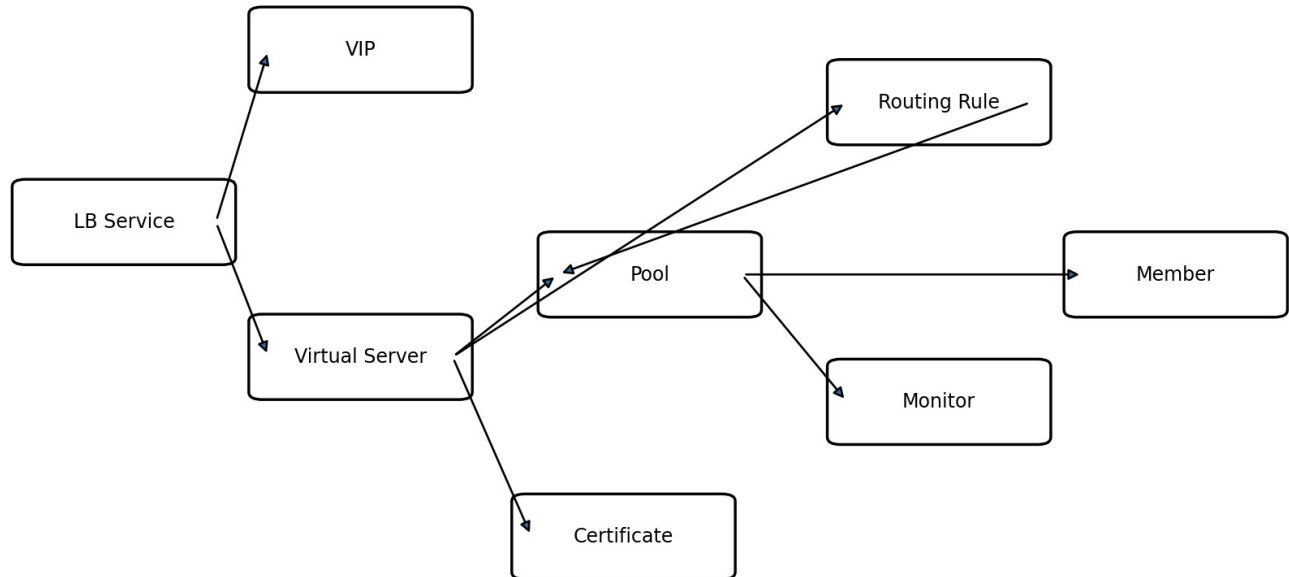


Figure 5-3. Dependency graph used for create/update and reverse-order deletion.

5.5.1 Action Types

Action	Meaning
NoOp	Observed normalized state already equals desired state.
Create	No owned match exists.
Update	Mutable fields differ.
Replace	Immutable identity or parent binding differs.
Delete	Owned observed child is no longer desired.
Adopt	Exact owner match exists but persisted provider ID is absent/stale.
Wait	An asynchronous parent operation must finish before a dependent action.
Blocked	Conflict, failed dependency, validation error, or unsafe ownership state.

5.5.2 Ordering Rules

- Create parents before children.
- Create pools before routing rules that reference them.
- Upload certificates before binding them to a Virtual Server.

- For replacement, create and verify the replacement before removing the old resource when CMP permits coexistence.
- Delete routing rules and bindings before deleting their pools or certificates.
- Delete the LB Service only after all owned children are absent.

```
Plan(desired, observed):
  nodes = classify each desired and observed resource
  add dependency edges
  reject cycles or unresolved references
  topologically sort create/update actions
  reverse-topologically sort deletes
  insert wait actions for asynchronous dependencies
  return immutable DeploymentPlan
```

5.6 Typed CMP Client Architecture

The typed client layer maps one-to-one to CMP endpoint families. It hides URLs, headers, serialization, response envelopes, status-code interpretation, and pagination. It does not decide whether an object should exist.

```
type CMPClients struct {
  LBServices      LBServiceClient
  VIPs            VIPClient
  VirtualServers  VirtualServerClient
  Pools           PoolClient
  Members         PoolMemberClient
  Monitors        MonitorClient
  RoutingRules    RoutingRuleClient
  Certificates    CertificateClient
  Operations      OperationClient
}

type LBServiceClient interface {
  List(ctx context.Context, scope TenantScope, filter LBServiceFilter) ([]LBService, error)
  Get(ctx context.Context, scope TenantScope, id string) (LBService, error)
  Create(ctx context.Context, scope TenantScope, req CreateLBServiceRequest, meta RequestMeta)
  (OperationRef, error)
  Delete(ctx context.Context, scope TenantScope, id string, meta RequestMeta) (OperationRef,
  error)
}
```

5.6.1 Transport Responsibilities

- Ce-Auth/HMAC signing and timestamp generation.
- Correlation ID and idempotency/request key headers where supported.
- Per-request deadlines and context cancellation.
- JSON encoding/decoding with strict error handling.
- Response size limits and safe body closure.
- Redaction of credentials, private keys, and authorization material.
- Metrics for endpoint, method, status class, latency, and retry count.

5.6.2 Retry Boundary

Important: The transport may automatically retry only requests proven safe at the HTTP level, such as a GET or a mutation carrying a provider-supported idempotency key. Business retries

after 409, 422, unknown outcome, or asynchronous failure belong to managers and require rediscovery.

5.7 Typed API Models and Mapping

Three model families remain deliberately separate: internal desired models, typed CMP request/response DTOs, and normalized observed models. This prevents provider schema changes from leaking into controller logic.

Model family	Example	Purpose
Internal desired model	model.VirtualServer	Stable architecture contract produced by builders.
CMP request DTO	cmp.CreateVirtualServerRequest	Exact JSON schema for one endpoint.
CMP response DTO	cmp.VirtualServerResponse	Provider response including IDs and operation state.
Normalized observed model	deploy.ObservedVirtualServer	Comparable representation after defaults and volatile fields are removed.

```

type CreatePoolMemberRequest struct {
    ResourceID    string `json:"resource_id"`
    ResourceType  string `json:"resource_type"`
    BackendPortID string `json:"backend_port_id"`
    Address       string `json:"ip_address"`
    Port          int32  `json:"port"`
    Weight        int32  `json:"weight,omitempty"`
}

type CMPErrorEnvelope struct {
    Code      string `json:"code,omitempty"`
    Message   string `json:"message,omitempty"`
    Details   map[string]any `json:"details,omitempty"`
    RequestID string `json:"request_id,omitempty"`
}

```

5.7.1 Mapping Rules

- Mapping functions are pure and unit-tested.
- Unknown enum values are preserved as Unknown rather than silently mapped to a default.
- Provider-generated fields, timestamps, health, and task state never participate in desired-state equality.
- Sensitive request fields implement safe String methods or are never logged.
- Response envelope parsing tolerates documented variations but fails closed on malformed success responses.

5.8 Asynchronous CMP Operations

Asynchronous CMP Operation State Machine

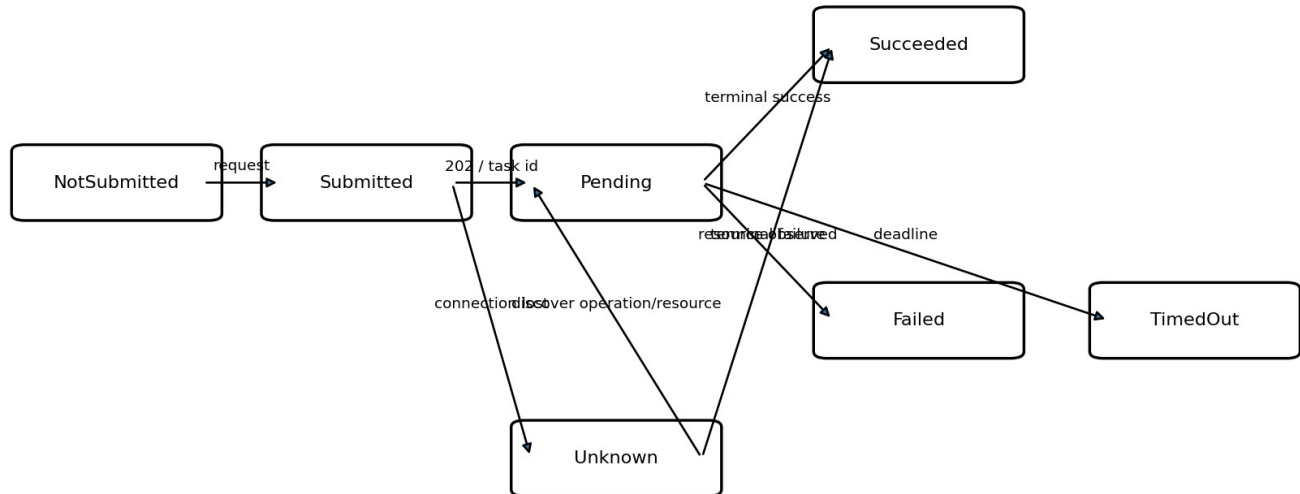


Figure 5-4. State machine for CMP operations that return before F5 completion.

A 202 response means the request was accepted for processing. The controller must not publish readiness or create dependent children until the required provider state is observed.

5.8.1 Operation Reference

```

type OperationRef struct {
    OperationID string
    ResourceID  string
    Kind        model.ResourceKind
    SubmittedAt time.Time
    RequestKey  string
    PollURL     string
}
  
```

5.8.2 Polling Algorithm

9. Honor Retry-After when supplied; otherwise use bounded exponential backoff with jitter.
10. Poll the operation endpoint when available; otherwise poll the resource endpoint and inspect provisioning state.
11. Treat terminal success only when CMP reports Active/Succeeded and required fields such as provider ID are present.
12. Treat terminal provider failure as a classified operation error containing the CMP request ID.
13. Stop polling when the reconcile context deadline is near; return RequeueAfter and resume in the next reconcile.
14. After a timeout or lost response, rediscover by request key or ownership key before retrying mutation.

Worker safety: Long provider operations should not hold a controller worker for many minutes. Persist enough operation identity in status/annotations or an internal operation store, return `RequeueAfter`, and continue polling in later reconciles.

5.9 Idempotency, Retries, and Ambiguous Outcomes

5.9.1 Idempotency Key

Every mutation receives a stable request key derived from cluster identity, owner UID, logical resource ID, desired generation/hash, and operation type. The same logical attempt reuses the same key across transport retries and process restarts.

```
requestKey = SHA256(
  clusterID + ownerUID + logicalID + desiredHash + operationType
)
```

5.9.2 Retry Matrix

Condition	Default response
Network failure before bytes are written	Retry within transport budget.
Connection lost after request may have reached CMP	Return <code>UnknownOutcome</code> ; rediscover before any new mutation.
HTTP 408/429/502/503/504	Retry with jitter and <code>Retry-After</code> , bounded by context and rate limiter.
HTTP 409	Rediscover and re-diff; may indicate concurrent operation or name conflict.
HTTP 401/403	Do not loop aggressively; classify Auth/Authorization and surface condition.
HTTP 400/422	Permanent until desired input changes; report validation/provider rejection.
Operation terminal failure	Classify provider code; retry only when explicitly transient.
Malformed success response	Treat as protocol error; do not assume success, rediscover.

5.9.3 Unknown Outcome Recovery

Recovery After an Ambiguous Timeout

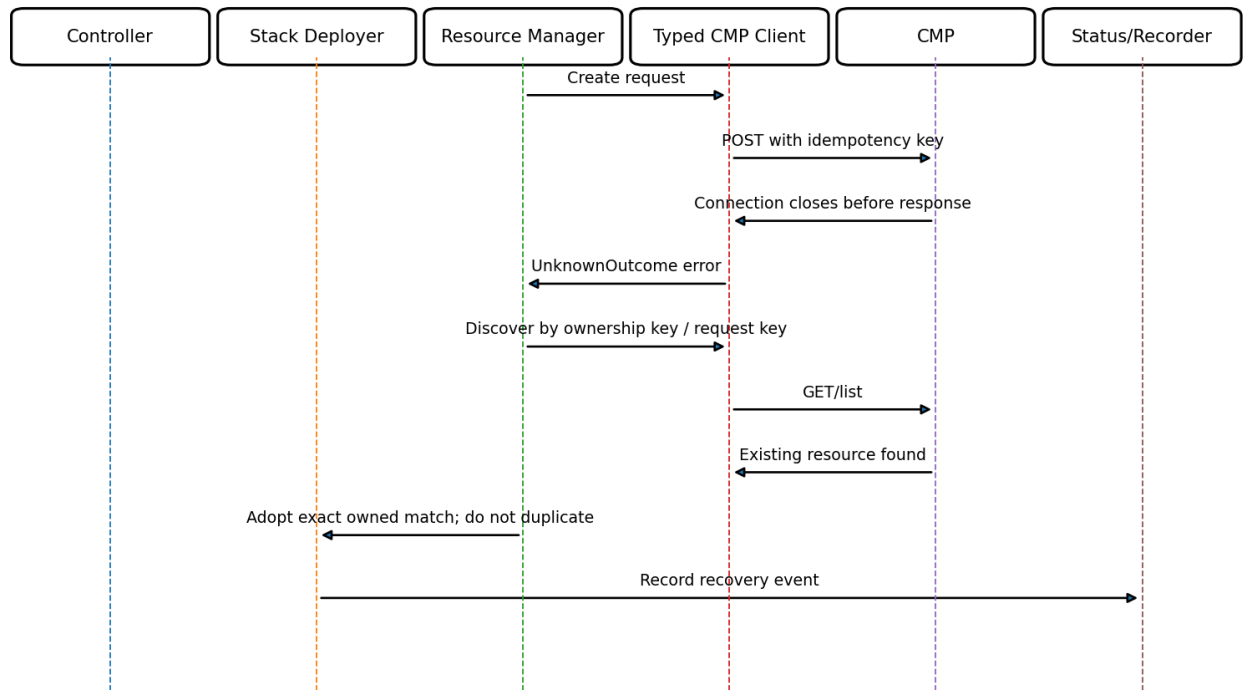


Figure 5-5. Rediscovery prevents duplication after a lost create response.

5.10 Discovery, Diffing, and Normalization

5.10.1 Discovery Sources

- Provider IDs persisted from a prior successful reconcile.
- Exact ownership metadata or deterministic ownership lookup key.
- Parent-scoped listing, for example pools under one Virtual Server.
- Name lookup only as a secondary hint; it never proves ownership by itself.

5.10.2 Normalization

Field type	Normalization
Provider defaults	Materialize the documented default in both desired and observed forms.
Ordering	Sort members, rules, certificates, and monitors by stable logical identity.
Names	Apply the same truncation, character replacement, and hash suffix algorithm.
Empty versus null	Canonicalize semantically equivalent values.
Protocol/enums	Normalize casing and aliases.
Volatile state	Exclude timestamps, task IDs, health samples, and provider-generated labels from equality.

Field type	Normalization
Secrets	Compare certificate fingerprint, never private-key bytes in logs or status.

```
func Equivalent(desired DesiredPool, observed ObservedPool) bool {
    d := NormalizeDesiredPool(desired)
    o := NormalizeObservedPool(observed)
    return cmp.Equal(d, o)
}
```

5.10.3 Drift Categories

Drift	Action
Mutable configuration drift	Patch/update.
Missing owned resource	Create/recreate.
Extra owned child	Delete after reference and ownership checks.
Immutable field drift	Replace according to planner.
Foreign resource collision	Block and report OwnershipConflict.
Provider-only health degradation	Do not overwrite desired config; report readiness and retry verification.

5.11 Ownership Identity and Metadata

Ownership Identity and Adoption Decision

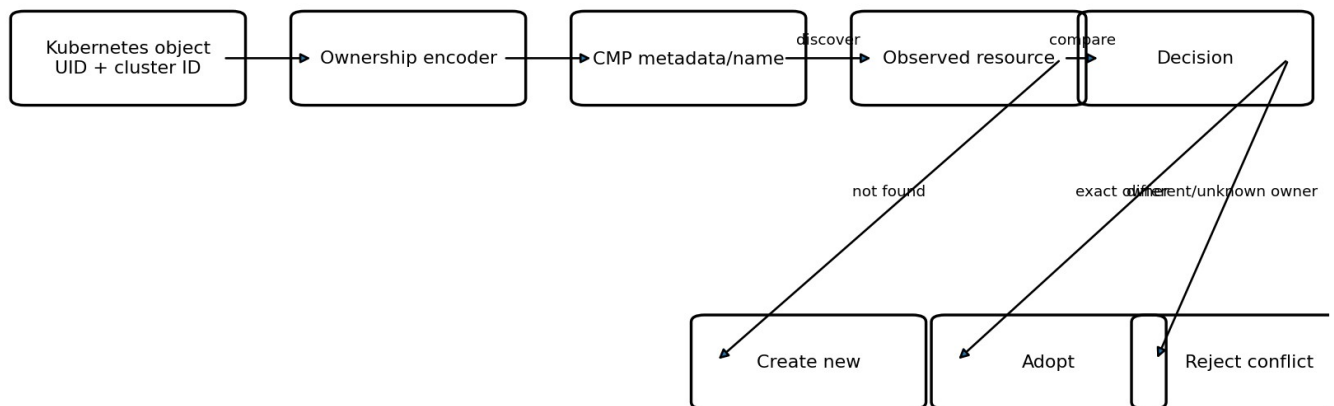


Figure 5-6. Ownership encoding, discovery, and decision points.

Ownership prevents two clusters, controllers, Services, or Ingress groups from controlling the same CMP resource. The owner identity is generated by the extension; it is not supplied by an application user.

5.11.1 OwnerRef

```
type OwnerRef struct {
    ClusterID string
    Kind       string           // Service or IngressGroup
    Namespace  string
    Name       string
    UID        types.UID
    GroupKey   string
    Controller string
}
```

5.11.2 Required Ownership Fields

Field	Purpose
managed-by	Stable controller identifier.
cluster-id	Separates resources from different Shoot clusters.
owner-kind	Service or IngressGroup.
owner-uid	Strong Kubernetes lifecycle identity; name reuse does not inherit ownership.
logical-id	Stable identity of the child inside the stack.
desired-hash	Optional drift and operation diagnostic.
source-namespace/name	Human-readable troubleshooting only; not sufficient for proof.

5.11.3 Storage Strategy

Preferred order: structured CMP tags/metadata; explicit description fields if queryable; deterministic names containing a short ownership hash as a fallback. The implementation should use the strongest available mechanism for each CMP resource type and record the strategy in the observed model.

Name safety: Display names may include readable prefixes, but ownership verification must use owner UID and cluster identity encoded in metadata or a collision-resistant hash. Namespace/name alone is unsafe because Kubernetes objects can be deleted and recreated.

5.12 Adoption and Conflict Handling

5.12.1 Adoption Rules

Observed situation	Decision
Persisted ID resolves and owner matches	Continue managing.
Persisted ID missing; exact ownership-key match found	Adopt and refresh persisted ID.

Observed situation	Decision
One exact owner match with stale desired hash	Adopt, then reconcile drift.
Multiple exact-looking matches	Block; manual investigation required.
Same name but no ownership metadata	Do not adopt automatically.
Different cluster ID or owner UID	Foreign resource conflict; never mutate.
Legacy resource with approved migration marker	Adopt only through explicit migration mode and audit event.

5.12.2 Adoption Preconditions

- Resource kind and parent chain match the desired logical resource.
- Tenant/project/network scope matches.
- Owner UID and cluster identity match exactly, or a controlled legacy migration rule applies.
- No second candidate exists.
- Adoption will not steal a resource referenced by another managed graph.

5.12.3 Conflict Status

Ownership conflicts are permanent until the external state or desired identity changes. The controller sets a clear condition such as Ready=False, Reason=OwnershipConflict, emits a Warning event, and does not continuously hammer CMP with mutations.

5.13 Validation Before Mutation

Deployment Validation Pipeline

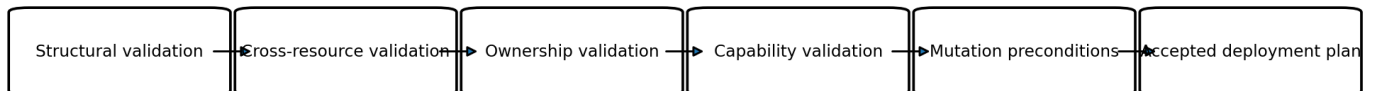


Figure 5-7. Deployment-layer validation performed after model building and before CMP mutation.

Stage	Checks
Structural	Required logical IDs, parent references, unique child identities, valid port ranges.
Cross-resource	Every rule target pool exists; certificate binding targets HTTPS listener; member parent exists.
Ownership	OwnerRef complete; logical IDs deterministic; no foreign observed match selected.
Capability	CMP endpoint and feature support; count and payload limits; mutable versus immutable fields.

Stage	Checks
Mutation preconditions	Credentials available; tenant/project/network scope resolved; dependencies terminal and healthy enough.
Delete preconditions	Expected owner, expected parent, no remaining desired/reference dependency, fresh read performed.

5.13.1 Validation Result

```
type ValidationResult struct {
    Errors    []FieldError // blocks all mutation
    Warnings  []Warning    // may continue, surfaced in status/events
}
```

Fail closed: When ownership or parent identity cannot be proved, the correct behavior is to stop mutation, not to guess. Availability may temporarily remain on the last-known-good graph while the new desired generation is reported as rejected.

5.14 Deletion Safety and Finalization

Deletion Safety Pipeline

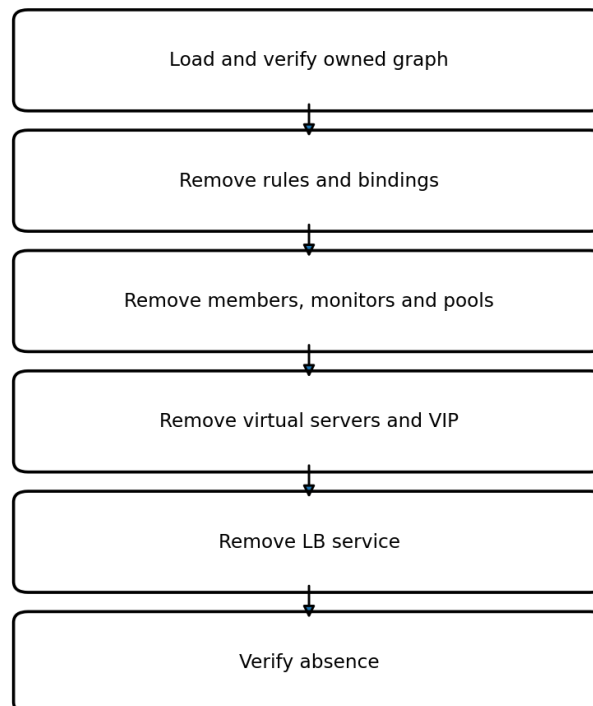


Figure 5-8. Reverse dependency deletion with repeated ownership verification.

5.14.1 Safety Rules

- Re-read each resource immediately before deletion.

- Verify owner UID, cluster ID, logical ID, resource kind, and expected parent.
- Delete only resources in the owner graph; never broad-delete by display-name prefix.
- Treat 404 as successful absence after the ownership decision has been made from prior state.
- Wait for asynchronous child deletion when CMP prevents parent deletion.
- Retain the Kubernetes finalizer until the provider graph is absent or a documented orphan policy is explicitly chosen.

5.14.2 Shared Resource Deletion

Ingress groups and shared frontend resources require graph-level reference counting from the desired group model. Removing one Ingress deletes only its rules, pools, members, monitors, or certificates that no remaining group member references. The LB Service, VIP, and listeners remain until the group is empty.

5.14.3 Stuck Deletion

Cause	Handling
CMP transient outage	Requeue with bounded backoff; finalizer remains.
Foreign child blocks parent deletion	Report OwnershipConflict; do not delete foreign child.
Credential revoked	Report AuthenticationFailed and alert; finalizer remains by default.
Provider resource manually removed	Verify 404/absence and continue cleanup.
Irrecoverable provider defect	Require explicit operator-approved orphan policy; never silently remove finalizer.

5.15 Error Classification

CMP Error Classification and Controller Response

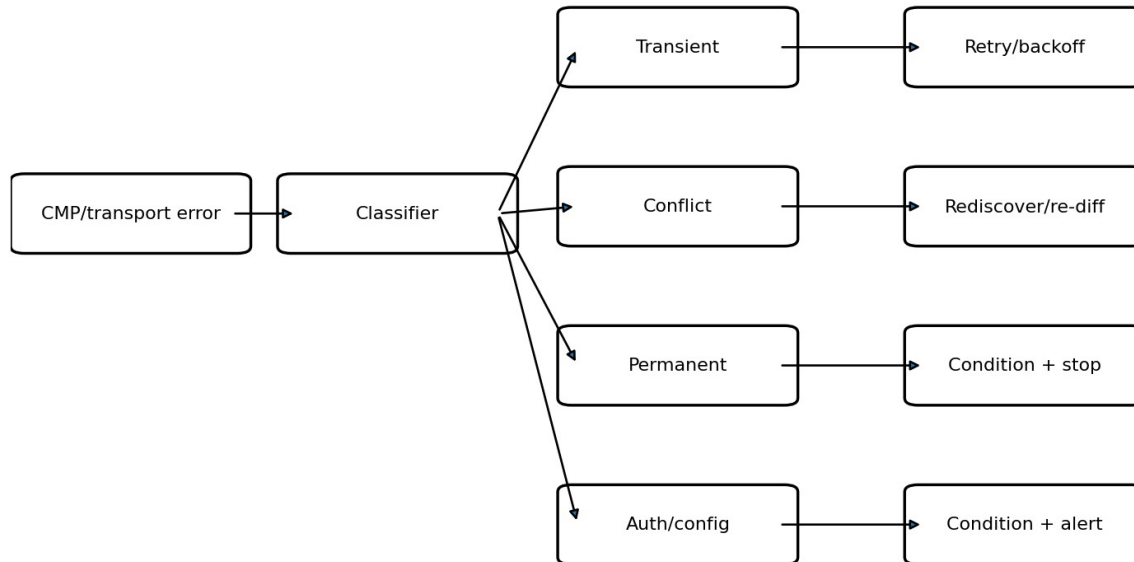


Figure 5-9. Error classes drive distinct reconcile behavior.

```

type ErrorClass string
const (
    ErrTransient      ErrorClass = "Transient"
    ErrThrottled      ErrorClass = "Throttled"
    ErrConflict        ErrorClass = "Conflict"
    ErrValidation      ErrorClass = "Validation"
    ErrAuthentication ErrorClass = "Authentication"
    ErrAuthorization   ErrorClass = "Authorization"
    ErrNotFound        ErrorClass = "NotFound"
    ErrUnknownOutcome ErrorClass = "UnknownOutcome"
    ErrProtocol        ErrorClass = "Protocol"
    ErrPermanent      ErrorClass = "Permanent"
)
  
```

Class	Examples	Controller behavior
Transient	502/503/504, connection reset, temporary provider failure.	Retry with jitter and rate limiting.
Throttled	429 or quota pacing response.	Honor Retry-After; reduce concurrency.
Conflict	409, operation already running, ownership/name collision.	Rediscover; retry only if convergence can resolve it.
Validation	400/422 caused by desired payload.	Set condition; wait for Kubernetes input change.
Authentication/Authorization	401/403.	Stop aggressive retries; alert and periodically recheck.

Class	Examples	Controller behavior
NotFound	404.	Context-dependent: absence on delete is success; absence on required parent triggers rebuild.
UnknownOutcome	Response lost after mutation may have executed.	Rediscover before retry.
Protocol	Malformed response, missing ID on success.	Rediscover and surface integration defect.
Permanent	Unsupported provider state or unrecoverable operation.	Condition and operator action.

5.15.1 Structured Provider Error

```

type ProviderError struct {
    Class      ErrorClass
    Operation  string
    Resource   model.LogicalID
    HTTPStatus int
    ProviderCode string
    RequestID  string
    RetryAfter time.Duration
    Cause      error
}

```

5.16 Status, Events, Metrics, and Auditability

5.16.1 Result Propagation

Signal	Contents
Kubernetes condition	Ready/Programmed, reason, concise message, observedGeneration.
Event	Important transition: created, adopted, conflict, async failure, deletion blocked.
Structured log	Owner, logical ID, provider ID, operation, request ID, elapsed time, error class.
Metric	Requests, latency, operations pending, retries, conflicts, resources, reconcile duration.
Trace	Controller reconcile -> manager action -> CMP request/poll chain.
Audit record	Adoption, ownership conflict, orphan override, destructive replacement.

5.16.2 Logging Rules

- Use correlation IDs across initial mutation and subsequent polls.
- Never log Ce-Auth secrets, private keys, full certificate bodies, tokens, or unredacted request headers.
- Log provider request IDs because they are essential for CMP support diagnostics.

- Avoid logging complete backend lists at info level; use counts and hashes, with sampled debug details.

5.17 Concurrency, Rate Limiting, and Caching

5.17.1 Concurrency Model

- Serialize deployment for the same OwnerRef using keyed locking or controller workqueue guarantees.
- Allow bounded parallelism across unrelated owners.
- Within one stack, parallelize independent branches only after their parents are ready.
- Use a global CMP rate limiter plus endpoint-specific limits for expensive operations.
- Do not run two mutations concurrently against the same CMP resource or parent collection unless explicitly supported.

5.17.2 Cache Policy

Data	Cache recommendation
Static capability/configuration	Cache with version or TTL.
Tenant/project/network resolution	Short TTL; invalidate on authorization/not-found errors.
Operation result	Persist/recover; do not rely only on in-memory cache.
Resource lists	Use only within one reconcile snapshot; refresh before destructive action.
Credentials	Use secret-backed provider and short-lived memory; never persist plaintext.

5.18 Security and Credential Handling

- Use a dedicated CMP credential with minimum permissions for the target organization/project.
- Read credentials through a provider interface backed by Kubernetes Secret or Gardener extension secret plumbing.
- Sign each request at send time; avoid reusing stale timestamps/nonces.
- Use TLS verification with managed trust roots; no insecure-skip-verify in production.
- Redact all sensitive headers and payload fields in logging and tracing.
- Set strict request timeouts, response-size bounds, and JSON decoding limits.
- Validate all CMP IDs and URL path components before construction.
- Separate certificate private-key handling from normal DTO logging and metrics.

5.19 Testing Strategy

Test layer	Coverage
Pure unit tests	Normalization, mapping, ownership encoding,

Test layer	Coverage
	action classification, error classifier.
Manager unit tests	Create/update/no-op/replace/delete/adopt using fake typed clients.
Planner tests	Dependency order, cycles, shared references, reverse deletion, partial failures.
HTTP contract tests	Exact methods, paths, headers, JSON bodies, envelope parsing, pagination.
Async tests	202 polling, Retry-After, timeout, terminal failure, process restart, unknown outcome.
Fault injection	Dropped response after create, 429 storms, stale lists, malformed success body, duplicate candidate.
Integration tests	Mock CMP server with realistic lifecycle and eventual consistency.
End-to-end tests	Kubernetes Service/Ingress through extension into a test CMP/F5 environment.

5.19.1 Required Golden Scenarios

- Controller restart after CMP accepted create but before status write.
- Pool exists but one member and monitor are missing.
- Two resources share a display name but only one carries the correct owner UID.
- Delete is requested while CMP reports an operation already in progress.
- Certificate update requires replacement and rebinding without breaking unrelated hosts.
- A foreign routing rule references an owned pool; deletion must stop and report conflict.

5.20 Package and Interface Design

```

pkg/
  model/                # desired-state contracts from Part 2
  deployment/
    stack/              # planner and orchestration
      lb-service/
        vip/
          virtualserver/
            pool/
              member/
                monitor/
                  routingrule/
                    certificate/
                      ownership/
                        validation/
                          errors/
                            operation/
  cmp/
    clientset.go
    transport/
      auth/
        lb-service/
          vip/
            virtualserver/
              pool/
                member/

```

```

monitor/
routingrule/
certificate/
operation/
observability/

```

5.20.1 Key Interfaces

```

type OperationWaiter interface {
    Wait(ctx context.Context, ref OperationRef) (OperationResult, error)
}

type OwnershipVerifier interface {
    Verify(expected OwnerRef, observed ResourceMetadata) error
}

type Planner interface {
    Build(desired *model.Stack, observed *ObservedGraph) (*DeploymentPlan, error)
}

type ErrorClassifier interface {
    Classify(err error, response *http.Response) *ProviderError
}

```

5.21 CMP Endpoint Mapping

The following endpoint families were identified in the supplied CMP Compute Swagger. Exact request/response schemas remain encapsulated in typed clients so the rest of the extension does not depend on URL structure.

Resource	CMP endpoint family	Operations used
LB Service	/load-balancers/domain/ {organization_id}/project/ {project_id}/load-balancers/ lb_service/	GET list, POST create; GET/DELETE by lb_service_id.
VIP	.../lb_service/{lb_service_id}/ vip	GET list, POST create; DELETE by vip_id.
Virtual Server	.../load-balancers/ {lb_service_id}/virtual-servers	GET list, POST create; GET/PATCH/DELETE by virtual_server_id.
Routing Rule	.../lb_service/{lb_service_id}/ virtual-servers/ {virtual_server_id}/routing- rules	GET, POST; DELETE by routing_rule_id.
Certificate	.../load-balancers/ {lb_service_id}/certificates	GET, POST; DELETE by certificate_id; bind via Virtual Server certificate endpoint.
Pool	.../virtual-servers/ {virtual_server_id}/pools	POST; GET/DELETE by pool ID; set-default endpoint.
Pool Member	.../pools/{pool_id}/members	POST; PUT/DELETE by member ID; health and operating-status endpoints.
Monitor	.../pools/{pool_id}/monitors	GET/POST/PUT;

Resource	CMP endpoint family	Operations used
		PATCH/DELETE by monitor ID.
Health	Virtual Server and member health-check endpoints.	GET for readiness/diagnostics, not desired-state ownership.

Version isolation: Endpoint versioning, response envelopes, and future CMP schema changes are localized to cmp/* packages. Deployment managers consume stable domain-level client interfaces.

5.22 Sequence Diagrams

Asynchronous Create or Update Sequence

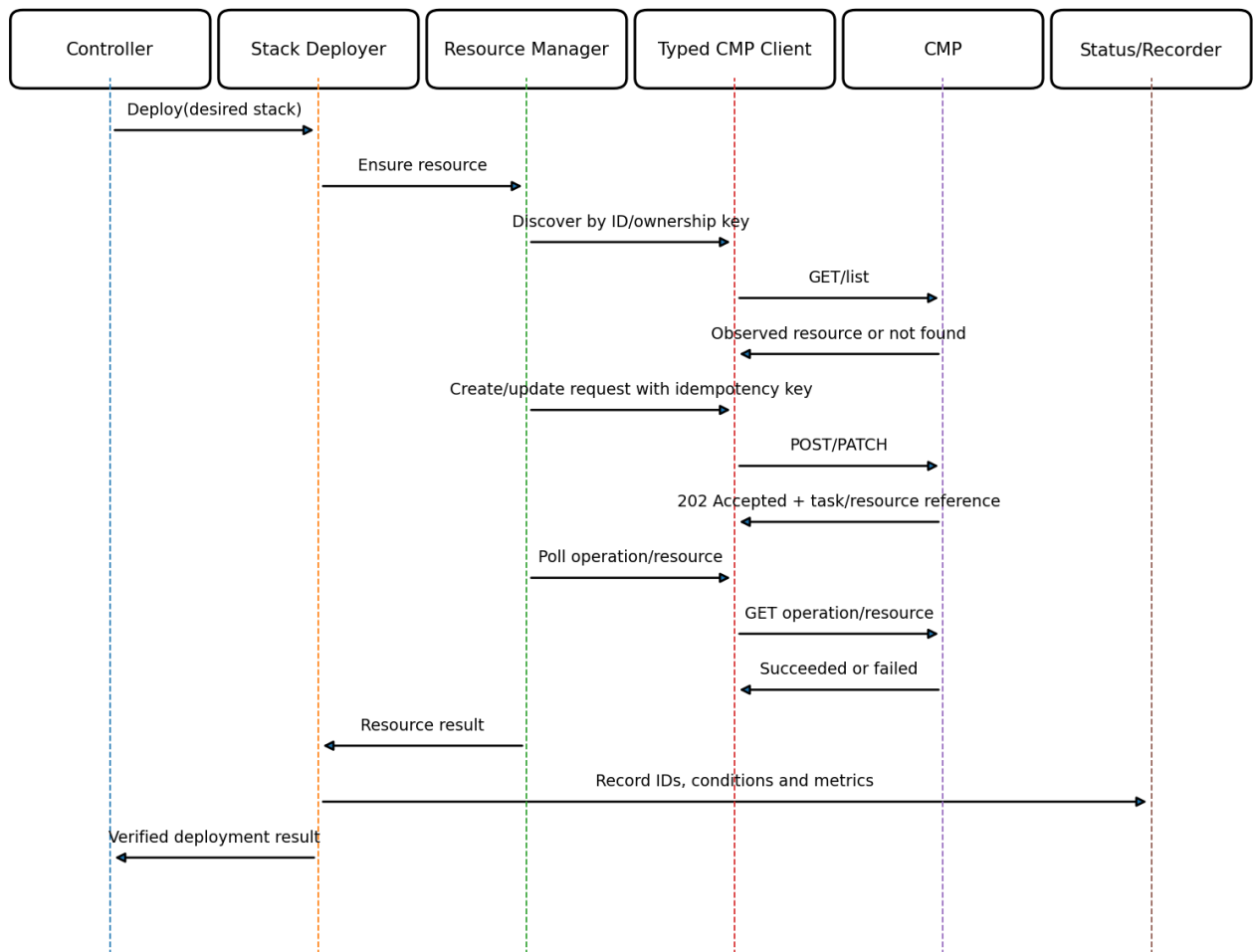


Figure 5-10. End-to-end asynchronous create/update with verification.

Safe Delete Sequence

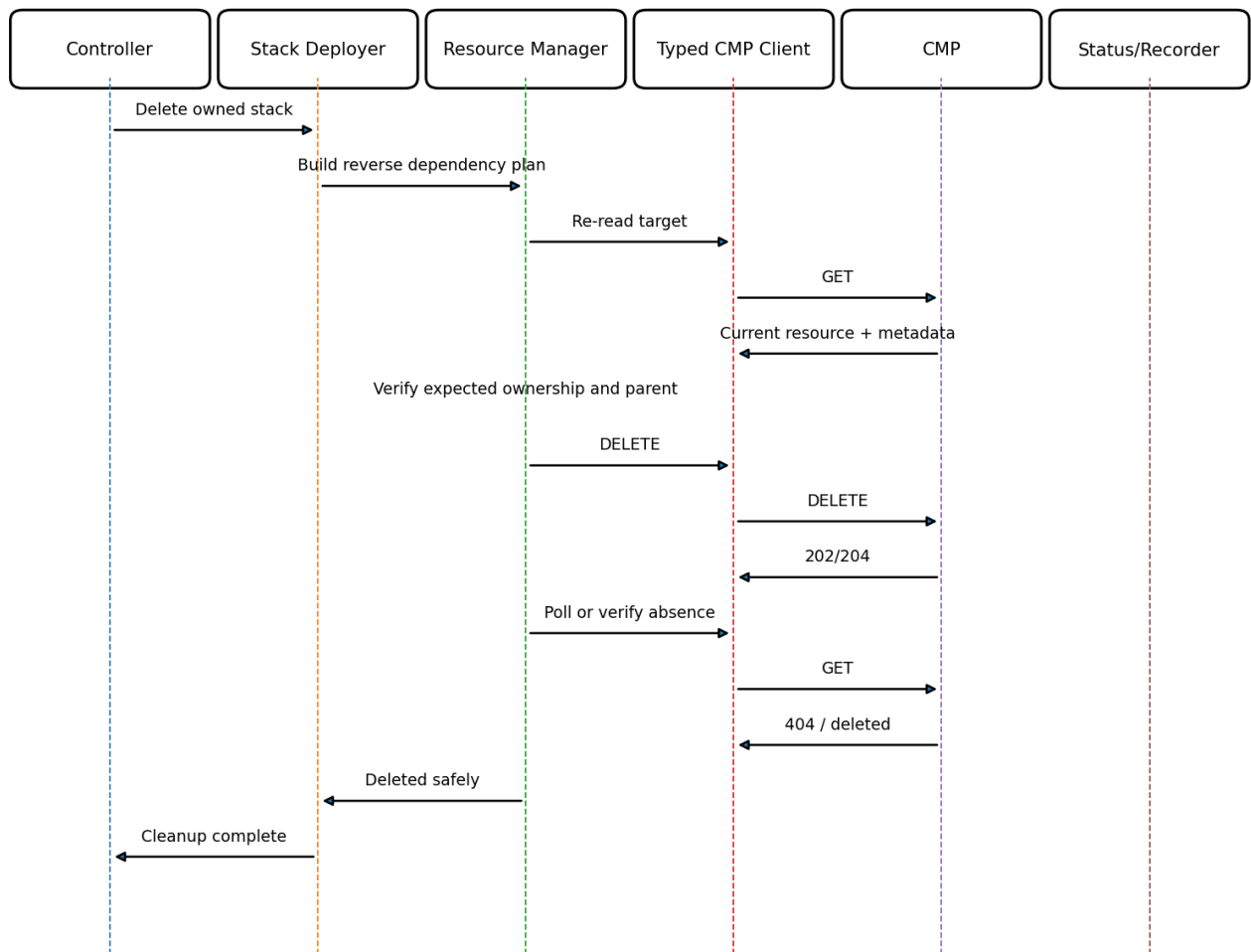


Figure 5-11. Safe deletion re-verifies ownership immediately before mutation.

5.23 Design Decisions

Decision	Rationale
One stack deployer shared by Service and Ingress	Avoids divergent provider behavior and duplicate safety logic.
Resource-specific managers	CMP lifecycle and update semantics differ by resource kind.
Typed clients below managers	Separates transport/API schema from reconciliation policy.
Desired and observed normalization	Prevents endless updates caused by defaults, ordering, or volatile fields.
Owner UID + cluster identity	Protects against Kubernetes name reuse and cross-cluster collisions.
Adoption requires exact proof	Availability is less important than avoiding resource theft or corruption.

Decision	Rationale
Async operations resume across reconciles	Prevents blocked workers and supports restart recovery.
Unknown outcome triggers discovery	Blind retries can create duplicate provider resources.
Deletion in reverse dependency order	Matches CMP referential constraints and minimizes partial failures.
Error classification is first-class	Different failures require different retry and user-visible behavior.

5.24 Implementation Checklist

Foundation

- ☐ Define domain client interfaces and typed DTOs.
- ☐ Implement transport, Ce-Auth/HMAC, correlation IDs, redaction, deadlines.
- ☐ Implement structured ProviderError and classifier.

Ownership

- ☐ Define OwnerRef and deterministic logical IDs.
- ☐ Implement metadata/name encoding and exact verifier.
- ☐ Implement adoption decision matrix and conflict conditions.

Deployment

- ☐ Implement observed graph and normalization.
- ☐ Implement resource managers.
- ☐ Implement planner and dependency execution.
- ☐ Implement asynchronous operation waiter and resume state.

Safety

- ☐ Implement mutation precondition validation.
- ☐ Implement reverse-order deletion and fresh ownership read.
- ☐ Implement shared-resource reference handling.

Operations

- ☐ Add conditions, Events, metrics, traces, and provider request IDs.
- ☐ Add rate limiting, bounded concurrency, and credential refresh.

☐ Complete contract, fault-injection, integration, and end-to-end tests.

Completion criterion: Part 5 is complete when controllers can pass a desired stack to the deployment layer without knowing CMP URLs or payloads, and the layer can safely converge, recover, adopt, reject conflicts, and delete resources under restarts and partial provider failures.